COURSE NAME: DEEP LEARNING

COURSE CODE: CSA4708

NAME: THAMIZHARASAN S

REG NO: 192424087

Vector and Matrix Operations for Deep Learning

1. A hospital uses a deep learning model to classify medical images. A small grayscale image is represented by a 3 × 3 matrix of pixel intensities. Before classification, the image must be converted into a one-dimensional vector, and the total pixel intensity must be calculated.

Write a Python program to flatten the image matrix into a one-dimensional vector and calculate the sum of all pixel intensities.

Image matrix:

A = [

10 20 30

40 50 60

70 80 90]

AIM:

To write a Python program to convert a 3 × 3image matrix into a one-dimensional vector and calculate the sum of all pixel intensities.

PROCEDURE:

1. Import the NumPy library.
2. Create a 3 × 3matrix containing pixel intensities.
3. Use the flatten() function to convert the matrix into a one-dimensional vector.
4. Use the sum() function to calculate the total pixel intensity.
5. Display the original matrix, flattened vector, and total sum.

PROGRAM CODE:

```
import numpy as np

A = np.array([

   [10, 20, 30],

   [40, 50, 60],

   [70, 80, 90]

])

vector = A.flatten()

total = np.sum(A)

print("Original Image Matrix:")

print(A)

print("\nFlattened Vector:")

print(vector)

print("\nTotal Pixel Intensity:", total)
```

INPUT:

A= [10 40 70

20 50 80

30 60 90]

OUTPUT:

```
Original Image Matrix:
[[10 20 30]
 [40 50 60]
 [70 80 90]]

Flattened Vector:
[10 20 30 40 50 60 70 80 90]

Total Pixel Intensity: 450
```

RESULT:

The image matrix was successfully flattened into a one-dimensional vector, and the total pixel intensity was calculated as 450.

2. A deep learning model predicts student performance based on study hours, practice-test scores, and assignment scores. The input features are X= [4,6,8], the corresponding weights are W= [0.5,0.3,0.2], and the bias is b=2.

Write a Python program to calculate the predicted output by computing the dot product of the input and weight vectors and adding the bias.

AIM:

To write a Python program to calculate the predicted output of a deep learning model using the dot product of the input feature vector and weight vector, along with a bias.

PROCEDURE:

1. Import the NumPy library.
2. Define the input features $X = [4, 6, 8]$.
3. Define the weights $W = [0.5, 0.3, 0.2]$and bias $b = 2$.
4. Calculate the dot product of the input and weight vectors.
5. Add the bias to obtain the predicted output.
6. Display the dot product and predicted output.

PROGRAM CODE:

```
import numpy as np

X = np.array([4, 6, 8])

W = np.array([0.5, 0.3, 0.2])

b = 2

dot_product = np.dot(X, W)

prediction = dot_product + b

print("Input Features:", X)
```

```
print("Weights:", W)
print("Bias:", b)
print("Dot Product:", dot_product)
print("Predicted Output:", prediction)
```

INPUT:

- Input features: $X = [4, 6, 8]$
- Weights: $W = [0.5, 0.3, 0.2]$
- Bias: $b = 2$

OUTPUT:

```
print( Predicted Output: , prediction)

Input Features: [4 6 8]
Weights: [0.5 0.3 0.2]
Bias: 2
Dot Product: 5.4
Predicted Output: 7.4
```

RESULT:

The dot product was calculated successfully, and the predicted output of the model was 7.4.

3. A weather prediction system receives temperature and humidity measurements from two locations. The input matrix is

X = [

30 60

35 70]

The model uses the weight matrix

W = [

0.2 0.5

0.4 0.3]

Write a Python program to multiply the input matrix by the weight matrix and display the predicted output matrix for both locations.

AIM:

To write a Python program to multiply an input matrix by a weight matrix and display the predicted output matrix for two locations.

PROCEDURE:

1. Import the NumPy library.
2. Create the input matrix Xcontaining temperature and humidity measurements.
3. Create the weight matrix W.
4. Multiply the two matrices using np.dot().
5. Display the input matrix, weight matrix, and predicted output matrix.

PROGRAM CODE:

```
import numpy as np
X = np.array([
    [30, 60],
    [35, 70]
])
W = np.array([
    [0.2, 0.5],
    [0.4, 0.3]
])
output = np.dot(X, W)
print("Input Matrix X:")
print(X)
print("\nWeight Matrix W:")
print(W)
print("\nPredicted Output Matrix:")
print(output)
```

INPUT:

$$X = \begin{bmatrix} 30 & 60 \\ 35 & 70 \end{bmatrix}$$

$$W = \begin{bmatrix} 0.2 & 0.5 \\ 0.4 & 0.3 \end{bmatrix}$$

OUTPUT:

```
Input Matrix X:
[[30 60]
 [35 70]]

Weight Matrix W:
[[0.2 0.5]
 [0.4 0.3]]

Predicted Output Matrix:
[[30.  33. ]
 [35.  38.5]]
```

RESULT:

The matrix multiplication was performed successfully, and the predicted output matrix for both locations was obtained.

4. A handwriting recognition model adjusts its weights during training to reduce prediction errors. The current weight vector is W= [0.7,0.5,0.9], the gradient vector is G= [0.2,0.1,0.3], and the learning rate is 0.1.

Write a Python program to update the weights using the gradient descent rule and display the updated weight vector.

AIM:

To write a Python program to update the weights of a deep learning model using the gradient descent rule with a given learning rate.

PROCEDURE:

1. Import the NumPy library.
2. Initialize the current weight vector W.
3. Define the gradient vector $G$and learning rate $\alpha = 0.1$.
4. Apply the gradient descent update rule.
5. Display the original weights, gradient, and updated weights.

PROGRAM CODE:

```
import numpy as np

W = np.array([0.7, 0.5, 0.9])

G = np.array([0.2, 0.1, 0.3])

learning_rate = 0.1

updated_W = W - learning_rate * G

print("Original Weights:", W)

print("Gradient:", G)
```

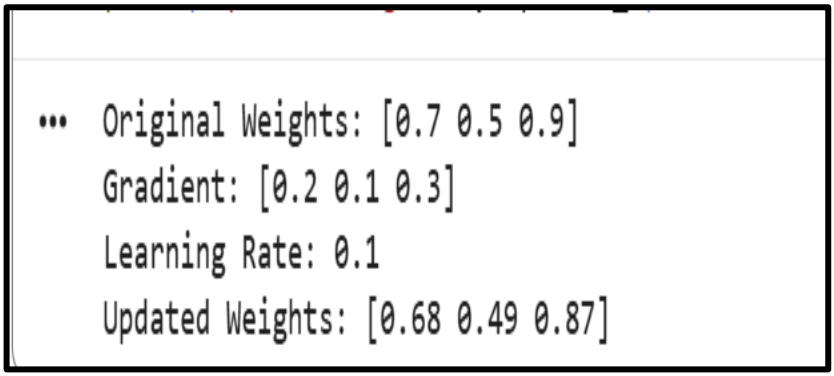
print("Learning Rate:", learning_rate)

print("Updated Weights:", updated_W)

INPUT:

- Weights: $W = [0.7, 0.5, 0.9]$
- Gradient: $G = [0.2, 0.1, 0.3]$
- Learning rate: $\alpha = 0.1$

OUTPUT:



RESULT:

The weights were successfully updated using gradient descent, producing the updated weight vector [0.68, 0.49, 0.87].

5. A recommendation system represents two movies using numerical feature vectors. The first movie is represented by A= [2,4,6], and the second movie is represented by B= [1,3,5]. The system combines the feature vectors to create a combined representation and calculates the similarity score using their dot product.

Write a Python program to perform vector addition and calculate the dot product of the two movie feature vectors. Display both results.

AIM:

To write a Python program to perform vector addition and calculate the dot product of two movie feature vectors to obtain a combined representation and similarity score.

PROCEDURE:

1. Import the NumPy library.
2. Define the first movie feature vector $A = [2, 4, 6]$.
3. Define the second movie feature vector $B = [1, 3, 5]$.
4. Perform vector addition using the + operator.
5. Calculate the dot product using np.dot().
6. Display the combined vector and dot product.

PROGRAM CODE:

```
import numpy as np

A = np.array([2, 4, 6])

B = np.array([1, 3, 5])

combined_vector = A + B

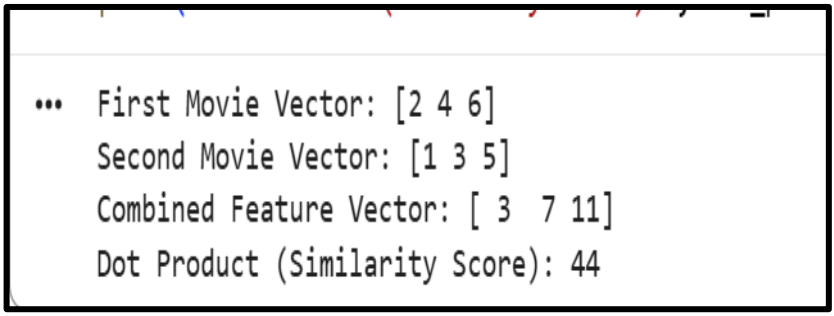
dot_product = np.dot(A, B)
```

```
print("First Movie Vector:", A)
print("Second Movie Vector:", B)
print("Combined Feature Vector:", combined_vector)
print("Dot Product (Similarity Score):", dot_product)
```

INPUT:

- First movie vector: $A = [2, 4, 6]$
- Second movie vector: $B = [1, 3, 5]$

OUTPUT:



RESULT:

Vector addition and the dot product were successfully calculated. The combined feature vector is [3, 7, 11], and the dot product is 44. The dot product is a similarity-related score, although normalized cosine similarity is generally more suitable when comparing vectors independently of their magnitudes.

Covariance, Eigenvalues and Eigenvectors Analysis

1. A college analyzes the relationship between students' study hours and examination marks to understand how study time is associated with academic performance. The study hours are [2, 4, 6, 8, 10], and the corresponding marks are [35, 45, 60, 75, 85].

Write a Python program to calculate the sample covariance between study hours and marks, and to draw a scatter plot showing their relationship.

AIM:

To write a Python program to calculate the sample covariance between students' study hours and examination marks and visualize their relationship using a scatter plot.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the study hours and examination marks.
3. Calculate the sample covariance using np.cov().
4. Display the covariance value.
5. Draw a scatter plot with study hours on the x-axis and marks on the y-axis.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

study_hours = np.array([2, 4, 6, 8, 10])

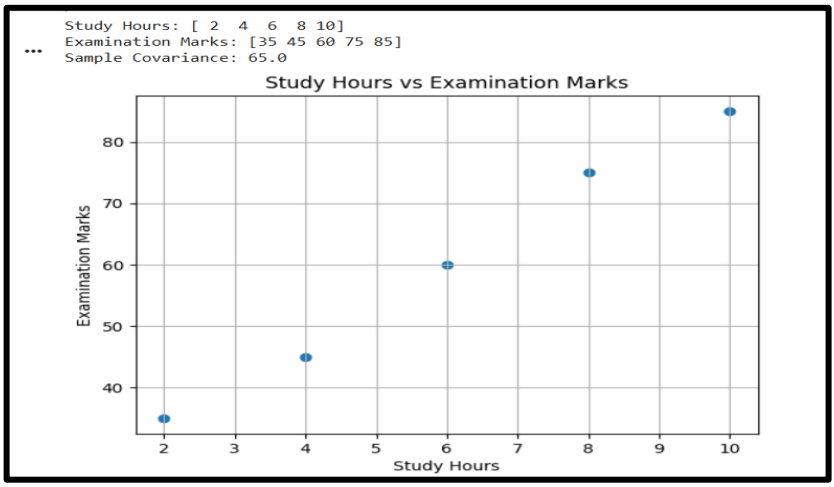
marks = np.array([35, 45, 60, 75, 85])
```

```
covariance = np.cov(study_hours, marks, ddof=1)[0, 1]
print("Study Hours:", study_hours)
print("Examination Marks:", marks)
print("Sample Covariance:", round(covariance, 2))
plt.scatter(study_hours, marks)
plt.xlabel("Study Hours")
plt.ylabel("Examination Marks")
plt.title("Study Hours vs Examination Marks")
plt.grid(True)
plt.show()
```

INPUT:

- Study hours: [2, 4, 6, 8, 10]
- Examination marks: [35, 45, 60, 75, 85]

OUTPUT:



RESULT:

The sample covariance was calculated as 65.0. The positive covariance indicates that study hours and examination marks tend to increase together.

2. A weather monitoring system records temperature and humidity over five days. The temperature values are [22, 25, 28, 30, 32], and the humidity values are [80, 75, 68, 60, 55].

Write a Python program to calculate the covariance matrix of temperature and humidity, and to visualize their relationship with a scatter plot.

AIM:

To calculate the covariance matrix of temperature and humidity measurements recorded over five days and visualize their relationship using a scatter plot.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the temperature and humidity arrays.
3. Combine the arrays into a data matrix.
4. Calculate the sample covariance matrix using np.cov().
5. Draw a scatter plot to show the relationship between temperature and humidity.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

temperature = np.array([22, 25, 28, 30, 32])

humidity = np.array([80, 75, 68, 60, 55])

data = np.vstack((temperature, humidity))

cov_matrix = np.cov(data, ddof=1)

print("Covariance Matrix:")
```

```
print(np.round(cov_matrix, 2))

plt.scatter(temperature, humidity)

plt.xlabel("Temperature (°C)")

plt.ylabel("Humidity (%)")

plt.title("Temperature vs Humidity")

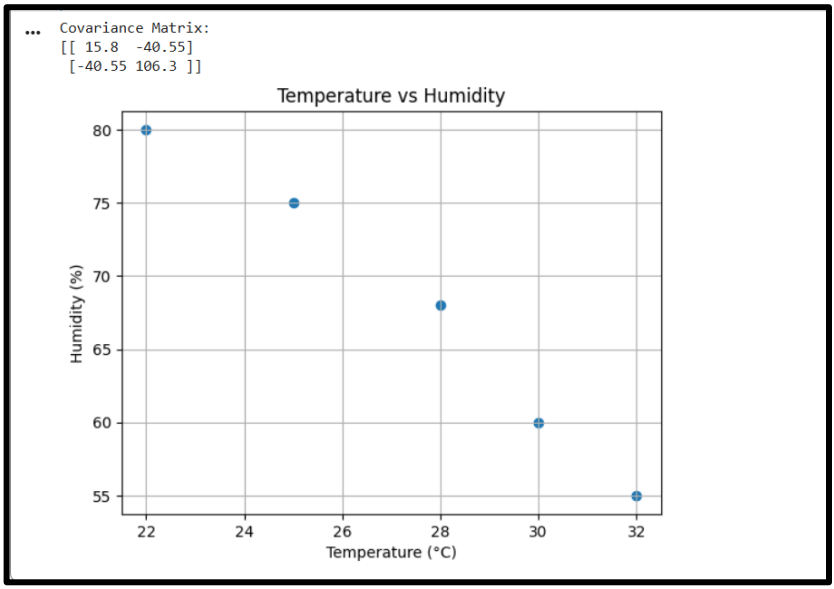
plt.grid(True)

plt.show()
```

INPUUT:

- Temperature: [22, 25, 28, 30, 32]
- Humidity: [80, 75, 68, 60, 55]

OUTPUT:



RESULT:

The covariance matrix was calculated successfully. The negative covariance of −39.75 indicates that higher temperatures are associated with lower humidity in the given observations.

3. An image-processing system uses the matrix below to analyze patterns in image features.

A = [

4 2

2 4]

Write a Python program to calculate the eigenvalues of the matrix and to display the eigenvalues in a bar chart.

AIM:

To write a Python program to calculate the eigenvalues of a given matrix and visualize them using a bar chart.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the given matrix A.
3. Calculate the eigenvalues using np.linalg.eigvals().
4. Display the eigenvalues.
5. Plot the eigenvalues in a bar chart.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

A = np.array([

   [4, 2],

   [2, 4]

])

eigenvalues = np.linalg.eigvals(A)
```

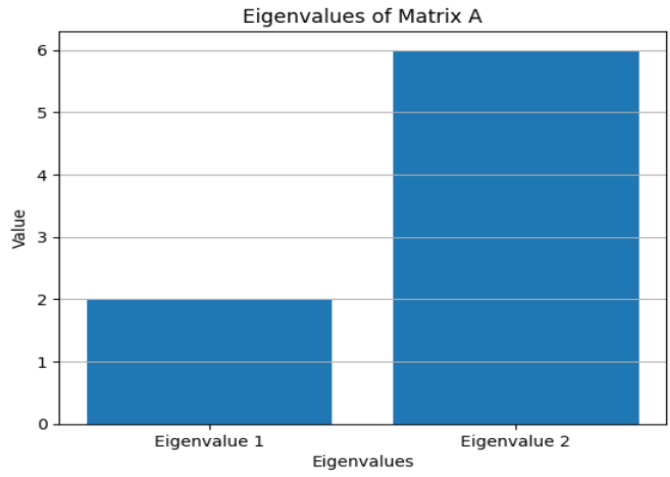
```
print("Input Matrix:")
print(A)
print("Eigenvalues:", np.sort(eigenvalues))
plt.bar(["Eigenvalue 1", "Eigenvalue 2"],
        np.sort(eigenvalues))
plt.xlabel("Eigenvalues")
plt.ylabel("Value")
plt.title("Eigenvalues of Matrix A")
plt.grid(axis="y")
plt.show()
```

INPUT:

$$W = \begin{bmatrix} 4 & 2 \\ 2 & 4 \end{bmatrix}$$

OUTPUT:

```
Input Matrix:
... [[4 2]
 [2 4]]
Eigenvalues: [2. 6.]
```



RESULT:

The eigenvalues of the given matrix were calculated successfully as 2 and 6 and displayed in a bar chart.

4. A machine learning model uses eigenvectors to identify important directions in a two-dimensional dataset. The covariance matrix is

C = [

3 1

1 3]

Write a Python program to calculate the eigenvalues and eigenvectors of the covariance matrix, and to plot the eigenvectors as arrows from the origin in a two-dimensional coordinate system.

AIM:

To calculate the eigenvalues and eigenvectors of a covariance matrix and visualize the eigenvectors as arrows originating from the origin in a two-dimensional coordinate system.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the covariance matrix C.
3. Calculate the eigenvalues and eigenvectors using np.linalg.eigh().
4. Display the eigenvalues and eigenvectors.
5. Use plt.quiver() to draw the eigenvectors as arrows from the origin.

PROGRAM CODE:

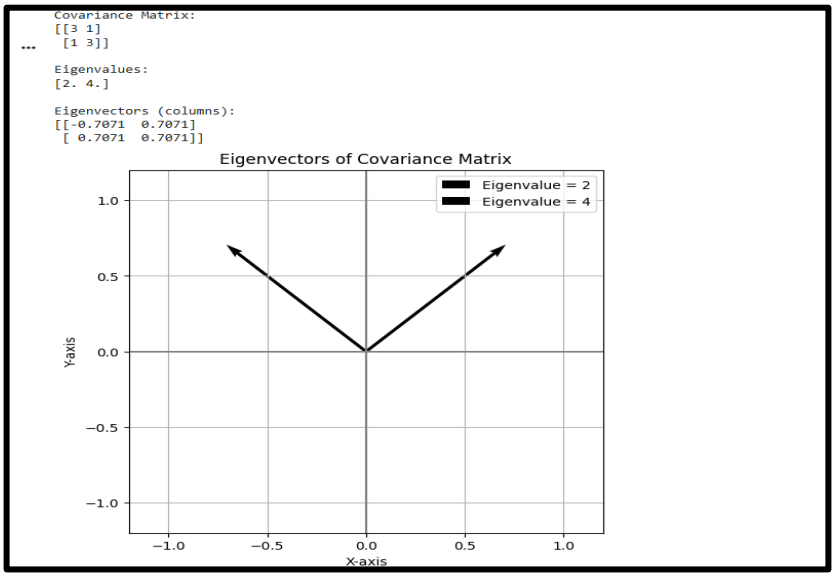
```
import numpy as np
import matplotlib.pyplot as plt
C = np.array([
    [3, 1],
    [1, 3]
])
eigenvalues, eigenvectors = np.linalg.eigh(C)
print("Covariance Matrix:")
print(C)
print("\nEigenvalues:")
print(eigenvalues)
print("\nEigenvectors (columns):")
print(np.round(eigenvectors, 4))
plt.figure(figsize=(6, 6))
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]
    plt.quiver(0, 0, v[0], v[1],
               angles="xy", scale_units="xy",
               scale=1, label=f"Eigenvalue = {eigenvalues[i]:.0f}")
plt.axhline(0, color="gray")
plt.axvline(0, color="gray")
plt.xlim(-1.2, 1.2)
```

```
plt.ylim(-1.2, 1.2)
plt.xlabel("X-axis")
plt.ylabel("Y-axis")
plt.title("Eigenvectors of Covariance Matrix")
plt.grid(True)
plt.legend()
plt.gca().set_aspect("equal")
plt.show()
```

INPUT:

$$W = \begin{bmatrix} 3 & 1 \\ 1 & 3 \end{bmatrix}$$

OUTPUT:



RESULT:

The eigenvalues were calculated as 2 and 4, and the corresponding eigenvectors were obtained. The arrows visualize the two principal directions associated with the covariance matrix.

5. An intelligent monitoring system collects measurements from two sensors. The covariance matrix of the measurements is

C = [

5 2

2 2]

The system uses eigenvalue and eigenvector analysis to identify the principal direction of variation in the sensor data.

Write a Python program using NumPy to calculate the eigenvalues and eigenvectors, identify the eigenvector corresponding to the largest eigenvalue, and use Matplotlib to visualize both eigenvectors as arrows on a two-dimensional graph.

AIM:

To calculate the eigenvalues and eigenvectors of a sensor covariance matrix, identify the eigenvector corresponding to the largest eigenvalue, and visualize both eigenvectors on a two-dimensional graph.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the covariance matrix C.
3. Calculate its eigenvalues and eigenvectors using np.linalg.eigh().
4. Identify the index of the largest eigenvalue using np.argmax().
5. Extract the corresponding eigenvector as the principal direction.

6. Plot both eigenvectors as arrows from the origin and highlight the principal direction in the legend.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
C = np.array([
    [5, 2],
    [2, 2]
])
eigenvalues, eigenvectors = np.linalg.eigh(C)
index = np.argmax(eigenvalues)
largest_eigenvalue = eigenvalues[index]
principal_eigenvector = eigenvectors[:, index]
print("Covariance Matrix:")
print(C)
print("\nEigenvalues:")
print(np.round(eigenvalues, 4))
print("\nEigenvectors (columns):")
print(np.round(eigenvectors, 4))
print("\nLargest Eigenvalue:", round(largest_eigenvalue, 4))
print("Principal Eigenvector:")
print(np.round(principal_eigenvector, 4))
plt.figure(figsize=(7, 7))
```

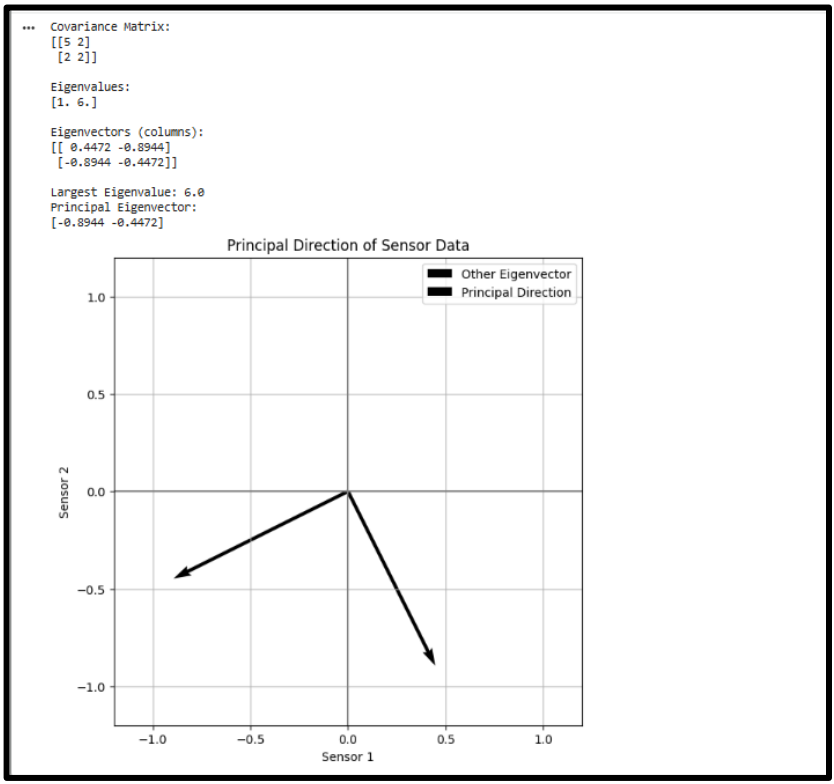
```
for i in range(len(eigenvalues)):
    v = eigenvectors[:, i]
    if i == index:
        label = "Principal Direction"
    else:
        label = "Other Eigenvector"
    plt.quiver(
        0, 0, v[0], v[1],
        angles="xy",
        scale_units="xy",
        scale=1,
        label=label
    )
plt.axhline(0, color="gray")
plt.axvline(0, color="gray")
plt.xlim(-1.2, 1.2)
plt.ylim(-1.2, 1.2)
plt.xlabel("Sensor 1")
plt.ylabel("Sensor 2")
plt.title("Principal Direction of Sensor Data")
plt.grid(True)
plt.legend()
plt.gca().set_aspect("equal")
```

plt.show()

INPUT:

$$W = \begin{bmatrix} 5 & 2 \\ 2 & 2 \end{bmatrix}$$

OUTPUT:



RESULT:

The eigenvalues were calculated as 1 and 6. The largest eigenvalue is 6, and its corresponding eigenvector, approximately [-0.8944, -0.4472], identifies the principal direction of variation in the sensor measurements. The opposite vector [0.8944, 0.4472] represents the same direction.

Probability Distributions and Bayes' Theorem

1. A factory produces electronic components, and the probability that a component is defective is 0.1. A quality inspector randomly selects 20 components. The number of defective components follows a binomial distribution.

Write a Python program to calculate the probability that exactly 3 components are defective, and to plot the probability mass function for 0 to 20 defective components.

AIM:

To calculate the probability that exactly 3 out of 20 randomly selected electronic components are defective and plot the binomial probability mass function (PMF).

PROCEDURE:

1. Import NumPy, Matplotlib, and the binomial distribution from SciPy.
2. Set the number of trials to 20 and the probability of a defective component to 0.1.
3. Calculate the probability of exactly 3 defective components using the binomial distribution.
4. Calculate the PMF for defective component counts from 0 to 20.
5. Plot the PMF using a bar chart.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import binom

n = 20
```

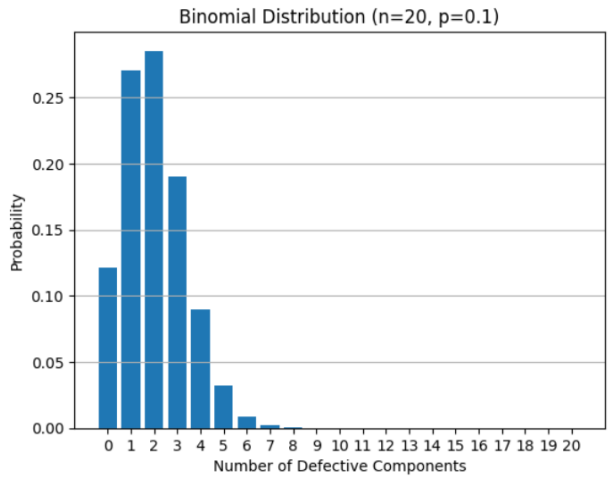
```
p = 0.1
k = 3
probability = binom.pmf(k, n, p)
print("Number of components:", n)
print("Probability of a defective component:", p)
print("Probability of exactly 3 defective components:",
    round(probability, 4))
x = np.arange(0, n + 1)
pmf = binom.pmf(x, n, p)
plt.bar(x, pmf)
plt.xlabel("Number of Defective Components")
plt.ylabel("Probability")
plt.title("Binomial Distribution (n=20, p=0.1)")
plt.xticks(x)
plt.grid(axis="y")
plt.show()
```

INPUT:

- Number of trials: $n = 20$
- Probability of a defective component: $p = 0.1$
- Required number of defective components: $k = 3$

OUTPUT:

```
Number of components: 20
Probability of a defective component: 0.1
Probability of exactly 3 defective components: 0.1901
```



RESULT:

The probability that exactly 3 components are defective is approximately 0.1901 (19.01%). The bar chart displays the probability distribution for all possible defective component counts from 0 to 20.

2. A lecturer observes that students' examination marks approximately follow a normal distribution with a mean of 70 and a standard deviation of 10.

Write a Python program to calculate the probability that a randomly selected student scores between 60 and 80, and to plot the normal distribution with the interval highlighted.

AIM:

To calculate the probability that a randomly selected student scores between 60 and 80 when examination marks follow a normal distribution with mean 70 and standard deviation 10, and visualize the interval.

PROCEDURE:

1. Import NumPy, Matplotlib, and the normal distribution from SciPy.
2. Define the mean as 70 and the standard deviation as 10.
3. Calculate the probability between 60 and 80 using the cumulative distribution function (CDF).
4. Generate values for the normal probability density function (PDF).
5. Plot the normal curve and shade the region between 60 and 80.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import norm
```

```
mean = 70
std_dev = 10
lower = 60
upper = 80
probability = (
    norm.cdf(upper, mean, std_dev)
    - norm.cdf(lower, mean, std_dev)
)
print("Mean:", mean)
print("Standard Deviation:", std_dev)
print("Probability of scoring between 60 and 80:",
      round(probability, 4))
print("Percentage:", round(probability * 100, 2), "%")
x = np.linspace(40, 100, 500)
y = norm.pdf(x, mean, std_dev)
plt.plot(x, y, label="Normal Distribution")
mask = (x >= lower) & (x <= upper)
plt.fill_between(x[mask], y[mask], alpha=0.4,
                 label="Scores between 60 and 80")
plt.xlabel("Examination Marks")
plt.ylabel("Probability Density")
plt.title("Normal Distribution of Examination Marks")
plt.legend()
```

```
plt.grid(True)

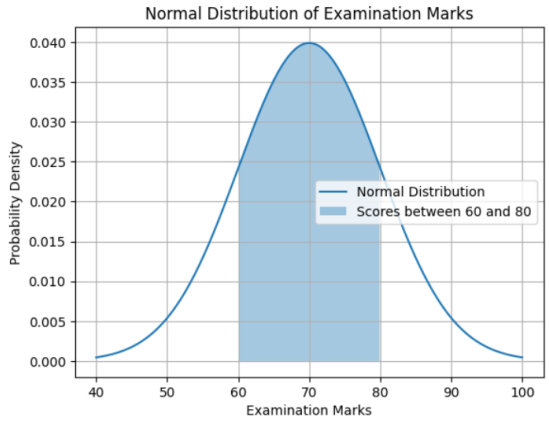
plt.show()
```

INPUT:

- Mean: $\mu = 70$
- Standard deviation: $\sigma = 10$
- Lower limit: 60
- Upper limit: 80

OUTPUT:

```
Mean: 70
Standard Deviation: 10
Probability of scoring between 60 and 80: 0.6827
Percentage: 68.27 %
```



RESULT:

The probability that a student scores between 60 and 80 is approximately 0.6827 (68.27%). The shaded area represents this probability under the normal distribution curve.

3. A service centre receives an average of 4 customers every 10 minutes. The number of customer arrivals follows a Poisson distribution.

Write a Python program to calculate the probability that exactly 6 customers arrive in a 10-minute interval, and to plot the probability mass function for customer arrivals from 0 to 12.

AIM:

To calculate the probability that exactly 6 customers arrive at a service centre in a 10-minute interval, given an average arrival rate of 4 customers, and plot the Poisson probability mass function for 0 to 12 arrivals.

PROCEDURE:

1. Import NumPy, Matplotlib, and the Poisson distribution from SciPy.
2. Set the average arrival rate to 4 customers per 10 minutes.
3. Calculate the probability of exactly 6 arrivals using the Poisson PMF.
4. Calculate the probabilities for customer arrivals from 0 to 12.
5. Plot the probability mass function using a bar chart.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from scipy.stats import poisson

lam = 4
```

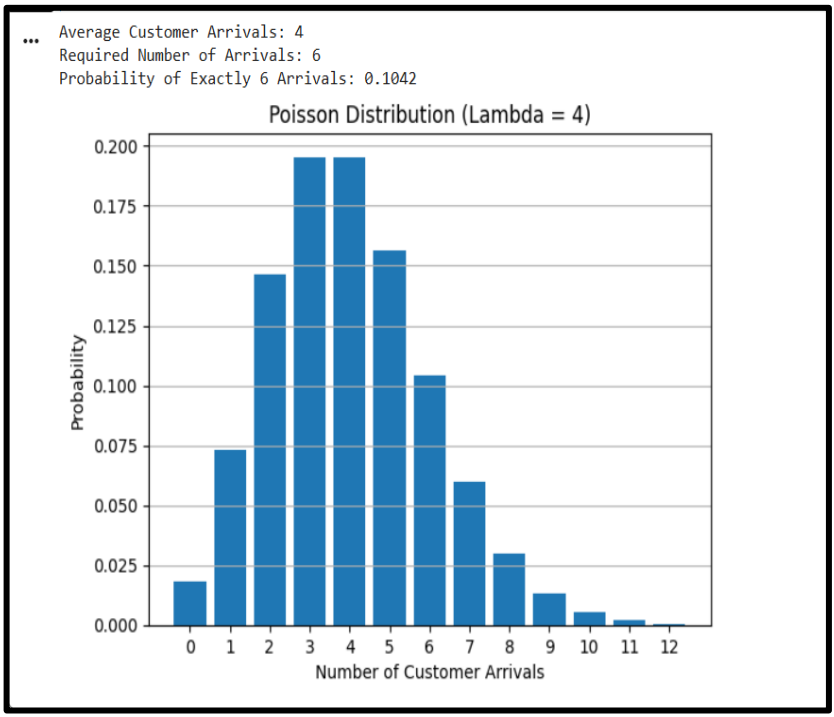
```
k = 6
probability = poisson.pmf(k, lam)

print("Average Customer Arrivals:", lam)
print("Required Number of Arrivals:", k)
print("Probability of Exactly 6 Arrivals:",
      round(probability, 4))
x = np.arange(0, 13)
pmf = poisson.pmf(x, lam)
plt.bar(x, pmf)
plt.xlabel("Number of Customer Arrivals")
plt.ylabel("Probability")
plt.title("Poisson Distribution (Lambda = 4)")
plt.xticks(x)
plt.grid(axis="y")
plt.show()
```

INPUT:

- Average arrival rate: $\lambda = 4$
- Required customer arrivals: $k = 6$
- Time interval: 10 minutes

OUTPUT:



RESULT:

The probability that exactly 6 customers arrive within 10 minutes is approximately 0.1042 (10.42%). The bar chart represents the probabilities of 0 to 12 customer arrivals.

4. A disease affects 2% of a population. A medical test correctly identifies a person with the disease 95% of the time. The test also produces a positive result for 5% of people who do not have the disease.

Write a Python program to apply Bayes' theorem and calculate the probability that a person actually has the disease given that the test result is positive. Use Matplotlib to compare the prior probability of having the disease with the posterior probability after a positive test result.

AIM:

To calculate the probability that a person actually has a disease given a positive medical test result using Bayes' theorem, and compare the prior and posterior probabilities using a bar chart.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the disease prevalence, test sensitivity, and false-positive rate.
3. Calculate the probability of a positive test result.
4. Apply Bayes' theorem to calculate the posterior probability.
5. Display the prior and posterior probabilities.
6. Plot a bar chart comparing the two probabilities.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

prior = 0.02

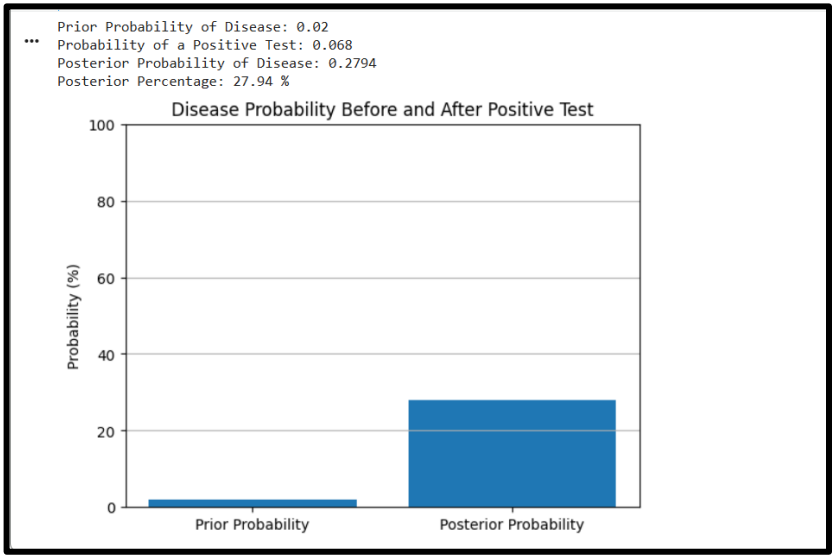
sensitivity = 0.95
```

```
false_positive_rate = 0.05
p_positive = (
    sensitivity * prior
    + false_positive_rate * (1 - prior)
)
posterior = (sensitivity * prior) / p_positive
print("Prior Probability of Disease:",
      round(prior, 4))
print("Probability of a Positive Test:",
      round(p_positive, 4))
print("Posterior Probability of Disease:",
      round(posterior, 4))
print("Posterior Percentage:",
      round(posterior * 100, 2), "%")
labels = ["Prior Probability", "Posterior Probability"]
values = [prior * 100, posterior * 100]
plt.bar(labels, values)
plt.ylabel("Probability (%)")
plt.title("Disease Probability Before and After Positive Test")
plt.ylim(0, 100)
plt.grid(axis="y")
plt.show()
```

INPUT:

- Prior probability of disease: $P(D) = 0.02$
- Test sensitivity: $P(+| D) = 0.95$
- False-positive rate: $P(+| \neg D) = 0.05$

OUTPUT:



RESULT:

Using Bayes' theorem, the probability that a person has the disease given a positive test result is approximately 27.94%, compared with the prior probability of 2%. This result illustrates why the prevalence of a disease and the false-positive rate matter when interpreting a positive medical test.

5. An email filtering system identifies 30% of incoming emails as spam. The word "prize" appears in 70% of spam emails and in 10% of non-spam emails.

Write a Python program to apply Bayes' theorem and calculate the probability that an email is spam given that it contains the word "prize." Use Matplotlib to visualize the prior probability of spam and the posterior probability after observing the word.

AIM:

To calculate the probability that an email is spam given that it contains the word "prize," using Bayes' theorem, and visualize the prior and posterior probabilities with a bar chart.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the prior probability of spam.
3. Define the probability that the word "prize" appears in spam and non-spam emails.
4. Calculate the overall probability that an email contains the word "prize."
5. Apply Bayes' theorem to obtain the posterior probability of spam.
6. Plot the prior and posterior probabilities for comparison.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

prior_spam = 0.30

word_in_spam = 0.70
```

```
word_in_nonspam = 0.10
p_word = (
    word_in_spam * prior_spam
    + word_in_nonspam * (1 - prior_spam)
)

posterior_spam = (
    word_in_spam * prior_spam
) / p_word

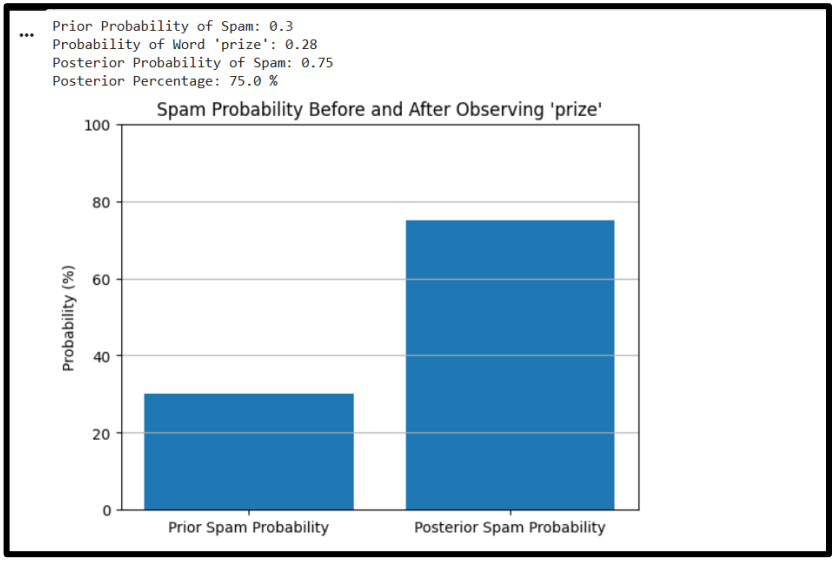
print("Prior Probability of Spam:",
      round(prior_spam, 4))
print("Probability of Word 'prize':",
      round(p_word, 4))
print("Posterior Probability of Spam:",
      round(posterior_spam, 4))
print("Posterior Percentage:",
      round(posterior_spam * 100, 2), "%")
labels = ["Prior Spam Probability",
          "Posterior Spam Probability"]
values = [prior_spam * 100, posterior_spam * 100]
plt.bar(labels, values)
plt.ylabel("Probability (%)")
```

```
plt.title("Spam Probability Before and After Observing 'prize'")
plt.ylim(0, 100)
plt.grid(axis="y")
plt.show()
```

INPUT:

- Prior probability of spam: $P(S) = 0.30$
- Probability of "prize" in spam: $P(W \mid S) = 0.70$
- Probability of "prize" in non-spam: $P(W \mid \neg S) = 0.10$

OUTPUT:



RESULT:

The posterior probability that an email is spam given that it contains the word "prize" is 75%, compared with the prior probability of 30%. Therefore, observing this word increases the estimated probability that the email is spam.

Entropy, Cross-Entropy and KL Divergence

1. A weather prediction system estimates the probabilities of three possible conditions as sunny, rainy, and cloudy: P= [0.6, 0.3, 0.1]. The system needs to measure the uncertainty in its predictions.

Write a Python program to calculate the entropy of the probability distribution and to display the probabilities of the three weather conditions in a bar chart.

AIM:

To calculate the entropy of a probability distribution representing sunny, rainy, and cloudy weather conditions and display their probabilities using a bar chart.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the weather probabilities.
3. Calculate entropy using the formula $H(P) = -\sum_i p_i \log(p_i)$.
4. Display the entropy value.
5. Plot the probabilities of the three weather conditions using a bar chart.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

P = np.array([0.6, 0.3, 0.1])

weather = ["Sunny", "Rainy", "Cloudy"]
```

```
entropy = -np.sum(P * np.log(P))

print("Weather Probabilities:", P)

print("Entropy:", round(entropy, 4), "nats")

plt.bar(weather, P)

plt.xlabel("Weather Conditions")

plt.ylabel("Probability")

plt.title("Weather Probability Distribution")

plt.ylim(0, 1)

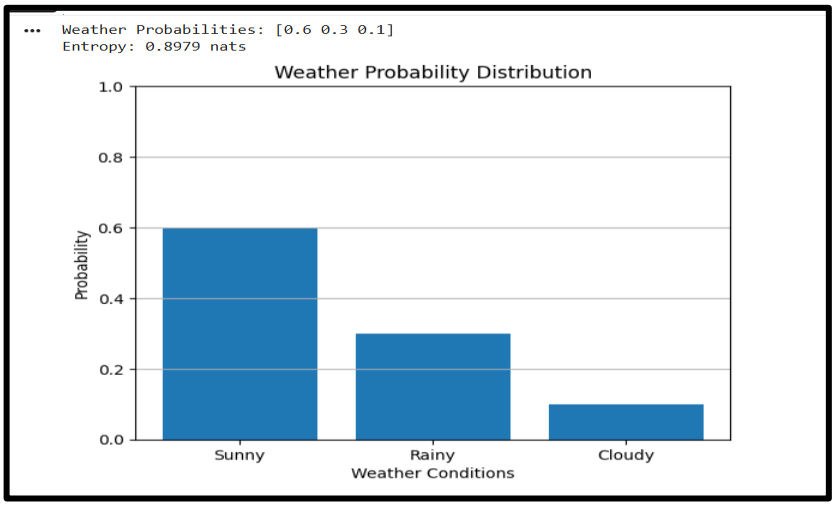
plt.grid(axis="y")

plt.show()
```

INPUT:

- Sunny: 0.6
- Rainy: 0.3
- Cloudy: 0.1

OUTPUT:



RESULT:

The entropy of the weather probability distribution was calculated as 0.8979 nats. The bar chart displays the probability of each weather condition. Entropy measures uncertainty in the distribution.

2. A deep learning model classifies an image into one of three categories: cat, dog, or bird.

The actual class is represented by Y= [1,0,0], while the model predicts probabilities P= [0.7,0.2,0.1].

Write a Python program to calculate the cross-entropy loss between the actual class distribution and the predicted probabilities, and to compare the actual and predicted probabilities.

AIM:

To calculate the cross-entropy loss between the actual class distribution and the predicted probabilities of a deep learning image classifier and compare the actual and predicted probabilities.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the actual class distribution $Y$and predicted probabilities P.
3. Calculate cross-entropy using $H(Y,P) = -\sum_i y_i \log(p_i)$.
4. Display the loss value.
5. Plot a grouped comparison of the actual and predicted class probabilities.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

Y = np.array([1, 0, 0])

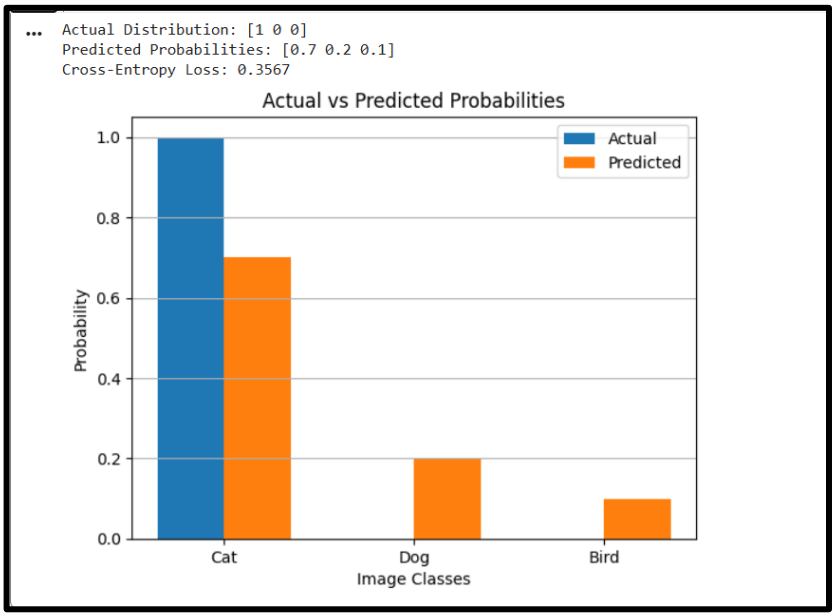
P = np.array([0.7, 0.2, 0.1])
```

```
classes = ["Cat", "Dog", "Bird"]
epsilon = 1e-15
P_safe = np.clip(P, epsilon, 1.0)
cross_entropy = -np.sum(Y * np.log(P_safe))
print("Actual Distribution:", Y)
print("Predicted Probabilities:", P)
print("Cross-Entropy Loss:", round(cross_entropy, 4))
x = np.arange(len(classes))
width = 0.35
plt.bar(x - width/2, Y, width, label="Actual")
plt.bar(x + width/2, P, width, label="Predicted")
plt.xticks(x, classes)
plt.xlabel("Image Classes")
plt.ylabel("Probability")
plt.title("Actual vs Predicted Probabilities")
plt.legend()
plt.grid(axis="y")
plt.show()
```

INPUT:

- Actual distribution: $Y = [1, 0, 0]$
- Predicted distribution: $P = [0.7, 0.2, 0.1]$

OUTPUT:



RESULT:

The cross-entropy loss was calculated as 0.3567 nats. The model assigns a probability of 0.7 to the correct class, cat. The grouped bar chart compares the actual one-hot distribution with the predicted probabilities.

3. Two language models predict the next word using the probability distributions P= [0.5,0.3,0.2] and Q= [0.4,0.4,0.2]. A developer wants to measure how much the second model's distribution differs from the first.

Write a Python program to calculate the KL divergence and to compare the two probability distributions with a grouped bar chart.

AIM:

To calculate the Kullback–Leibler (KL) divergence between two probability distributions and compare them using a grouped bar chart.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the probability distributions $P$and Q.
3. Calculate KL divergence using $D_{KL}(P \parallel Q) = \sum_i p_i \log(p_i/q_i)$.
4. Display the KL divergence value.
5. Plot a grouped bar chart to compare both distributions.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

P = np.array([0.5, 0.3, 0.2])

Q = np.array([0.4, 0.4, 0.2])

words = ["Word 1", "Word 2", "Word 3"]

kl_divergence = np.sum(P * np.log(P / Q))
```

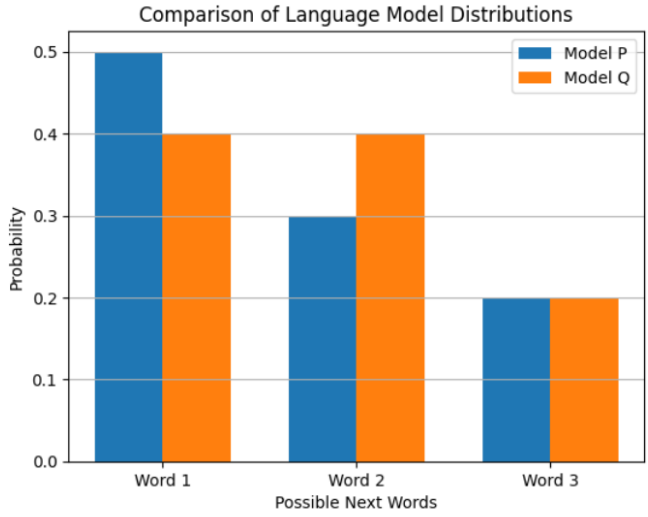
```
print("Distribution P:", P)
print("Distribution Q:", Q)
print("KL Divergence D_KL(P || Q):",
      round(kl_divergence, 4), "nats")
x = np.arange(len(words))
width = 0.35
plt.bar(x - width/2, P, width, label="Model P")
plt.bar(x + width/2, Q, width, label="Model Q")
plt.xticks(x, words)
plt.xlabel("Possible Next Words")
plt.ylabel("Probability")
plt.title("Comparison of Language Model Distributions")
plt.legend()
plt.grid(axis="y")
plt.show()
```

INPUT:

- Distribution P: [0.5, 0.3, 0.2]
- Distribution Q: [0.4, 0.4, 0.2]

OUTPUT:

```
...  Distribution P: [0.5 0.3 0.2]
     Distribution Q: [0.4 0.4 0.2]
     KL Divergence D_KL(P || Q): 0.0253 nats
```



RESULT:

The KL divergence $D_{KL}(P \parallel Q)$was calculated as 0.0253 nats. This small positive value indicates that the two probability distributions are relatively similar. KL divergence is directional, so reversing the distributions may produce a different result.

4. A medical decision-support system predicts whether a patient has a disease or not, with probabilities P= [0.9,0.1]. Another patient's predicted probabilities are Q= [0.5,0.5].

Write a Python program to calculate the entropy of both predictions and to compare their entropy values in a bar chart. Identify which prediction has greater uncertainty.

AIM:

To calculate and compare the entropy of two medical predictions and identify which prediction has greater uncertainty.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the two probability distributions $P = [0.9, 0.1]$and $Q = [0.5, 0.5]$.
3. Calculate the entropy of each distribution.
4. Display both entropy values.
5. Plot a bar chart comparing the entropy values.
6. Identify the prediction with greater uncertainty.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

P = np.array([0.9, 0.1])

Q = np.array([0.5, 0.5])

entropy_P = -np.sum(P * np.log(P))

entropy_Q = -np.sum(Q * np.log(Q))
```

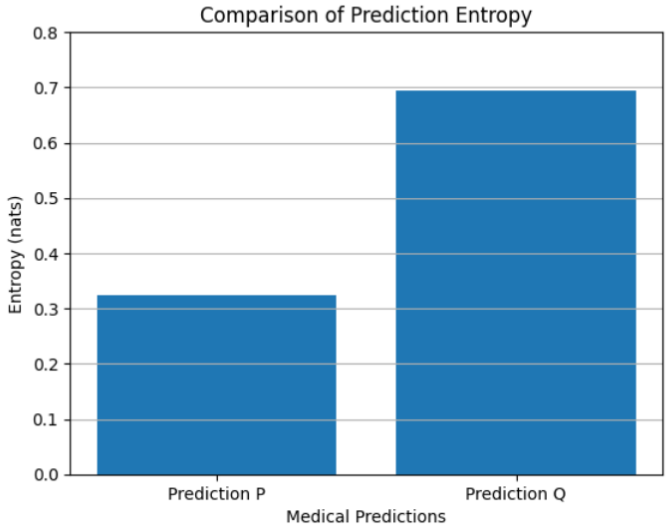
```
print("Prediction P:", P)
print("Prediction Q:", Q)
print("Entropy of P:", round(entropy_P, 4), "nats")
print("Entropy of Q:", round(entropy_Q, 4), "nats")
if entropy_P > entropy_Q:
    print("Prediction P has greater uncertainty.")
else:
    print("Prediction Q has greater uncertainty.")
labels = ["Prediction P", "Prediction Q"]
values = [entropy_P, entropy_Q]
plt.bar(labels, values)
plt.xlabel("Medical Predictions")
plt.ylabel("Entropy (nats)")
plt.title("Comparison of Prediction Entropy")
plt.ylim(0, 0.8)
plt.grid(axis="y")
plt.show()
```

INPUT:

- Prediction P: [0.9, 0.1]
- Prediction Q: [0.5, 0.5]

OUTPUT:

```
Prediction P: [0.9 0.1]
Prediction Q: [0.5 0.5]
Entropy of P: 0.3251 nats
Entropy of Q: 0.6931 nats
Prediction Q has greater uncertainty.
```



RESULT:

The entropy of prediction P is 0.3251 nats, while the entropy of prediction Q is 0.6931 nats. Therefore, prediction Q has greater uncertainty because both outcomes have equal probabilities. Prediction P is more confident in one outcome.

5. A neural network classifies an image into four categories. The actual class distribution is Y= [0,1,0,0], and the predicted probability distribution is P= [0.1,0.6,0.2,0.1]. The developer wants to evaluate the prediction error and compare the predicted distribution with the actual distribution.

Write a Python program to calculate the cross-entropy loss and KL divergence between the actual and predicted distributions, and to visualize the two distributions.

AIM:

To calculate the cross-entropy loss and KL divergence between an actual class distribution and a predicted probability distribution and visualize their comparison using a grouped bar chart.

PROCEDURE:

1. Import NumPy and Matplotlib.
2. Define the actual distribution $Y = [0, 1, 0, 0]$and predicted distribution $P = [0.1, 0.6, 0.2, 0.1]$.
3. Calculate the cross-entropy loss using the actual and predicted distributions.
4. Calculate KL divergence $D_{KL}(Y \parallel P)$.
5. Display both values.
6. Plot the actual and predicted distributions using a grouped bar chart.

PROGRAM CODE:

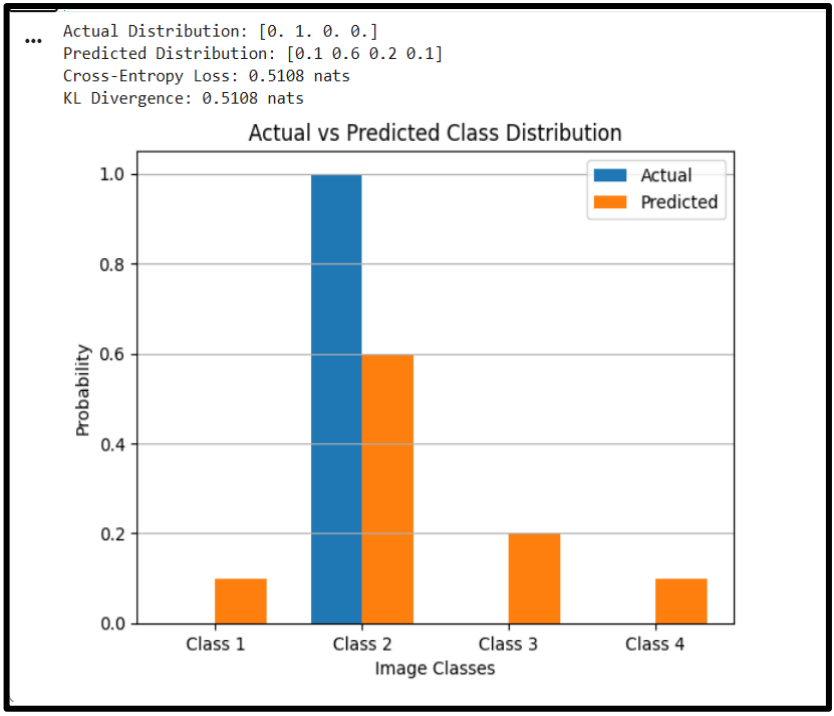
```
import numpy as np
import matplotlib.pyplot as plt
Y = np.array([0, 1, 0, 0], dtype=float)
P = np.array([0.1, 0.6, 0.2, 0.1], dtype=float)
classes = ["Class 1", "Class 2", "Class 3", "Class 4"]
epsilon = 1e-15
P_safe = np.clip(P, epsilon, 1.0)
cross_entropy = -np.sum(Y * np.log(P_safe))
mask = Y > 0
kl_divergence = np.sum(
    Y[mask] * np.log(Y[mask] / P_safe[mask])
)
print("Actual Distribution:", Y)
print("Predicted Distribution:", P)
print("Cross-Entropy Loss:", round(cross_entropy, 4), "nats")
print("KL Divergence:", round(kl_divergence, 4), "nats")
x = np.arange(len(classes))
width = 0.35
plt.bar(x - width/2, Y, width, label="Actual")
plt.bar(x + width/2, P, width, label="Predicted")
plt.xticks(x, classes)
plt.xlabel("Image Classes")
```

```
plt.ylabel("Probability")
plt.title("Actual vs Predicted Class Distribution")
plt.legend()
plt.grid(axis="y")
plt.show()
```

INPUT:

- Actual distribution: $Y = [0, 1, 0, 0]$
- Predicted distribution: $P = [0.1, 0.6, 0.2, 0.1]$

OUTPUT:



RESULT:

The cross-entropy loss and KL divergence were both calculated as 0.5108 nats. Since the actual distribution is one-hot encoded, its entropy is zero, making cross-entropy equal to KL divergence. The grouped bar chart shows that the model assigns a probability of 0.6 to the correct class.

Bias–Variance, Overfitting and Underfitting Analysis

1. A real estate company develops machine learning models to predict house prices based on house area. A linear regression model and polynomial regression models of degrees 2 and 10 are trained using the same dataset. The company wants to compare how model complexity affects prediction performance.

Write a Python program to generate sample house-price data, train polynomial regression models of degrees 1, 2, and 10, and to plot their prediction curves against the actual data.

AIM:

To generate sample house-price data, train polynomial regression models of degrees 1, 2, and 10, and compare their prediction curves with the actual data to understand the effect of model complexity.

PROCEDURE:

1. Import NumPy, Matplotlib, and Scikit-learn.
2. Generate synthetic house-area and house-price data with some random noise.
3. Split the data into training and testing sets.
4. Train polynomial regression models of degrees 1, 2, and 10.
5. Generate predictions over a smooth range of house areas.
6. Plot the actual data and the three fitted curves.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
np.random.seed(42)
area = np.linspace(500, 3000, 60)
price = (50 + 0.08 * area + 0.00002 * area**2
        + np.random.normal(0, 20, 60))
X = area.reshape(-1, 1)
y = price
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
x_curve = np.linspace(500, 3000, 300).reshape(-1, 1)
plt.figure(figsize=(10, 6))
plt.scatter(X, y, label="Actual Data")
for degree in [1, 2, 10]:
    model = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
```

```
        LinearRegression()
    )
    model.fit(X_train, y_train)

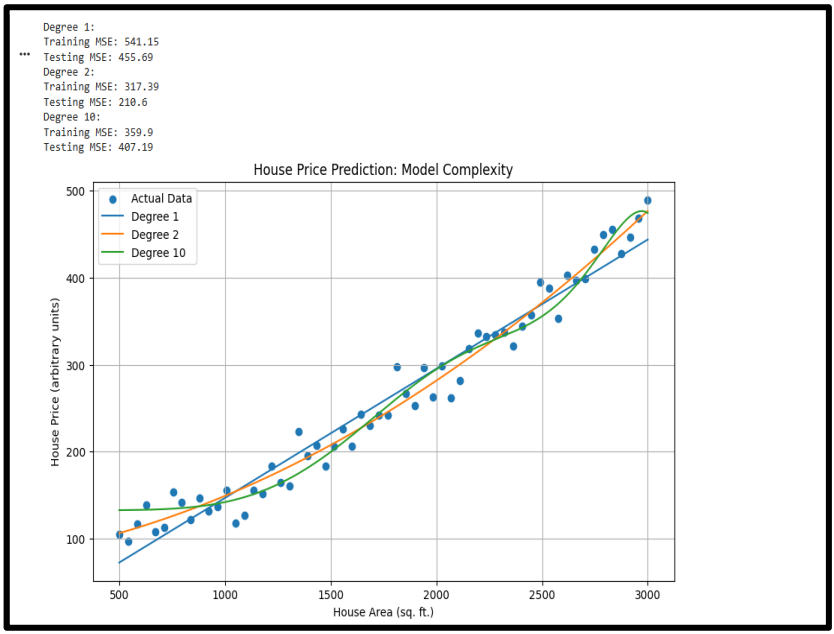
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    print(f"Degree {degree}:")
    print("Training MSE:", round(mean_squared_error(
        y_train, train_pred), 2))
    print("Testing MSE:", round(mean_squared_error(
        y_test, test_pred), 2))
    plt.plot(x_curve, model.predict(x_curve),
             label=f"Degree {degree}")
plt.xlabel("House Area (sq. ft.)")
plt.ylabel("House Price (arbitrary units)")
plt.title("House Price Prediction: Model Complexity")
plt.legend()
plt.grid(True)
plt.show()
```

INPUT:

- House areas: 60 generated values between 500 and 3000 square feet.

- House prices: A quadratic relationship with random noise.
- Polynomial degrees: 1, 2, and 10.
- Test size: 20%.

OUTPUT:



RESULT:

The polynomial models were trained and compared successfully. Degree 1 may underfit the nonlinear relationship, degree 2 can represent the underlying quadratic trend, and degree 10 can produce a more complex curve that may overfit the training data. The testing errors help determine which model generalizes best.

2. A college develops a machine learning model to predict students' examination marks based on study hours. The model uses a simple linear relationship, but the actual marks follow a nonlinear pattern.

Write a Python program using to generate nonlinear student-performance data, train a linear regression model, calculate its training and testing mean squared errors, and to compare the actual marks with the predicted values.

AIM:

To generate nonlinear student-performance data, train a linear regression model, calculate training and testing mean squared errors, and compare actual examination marks with predicted marks.

PROCEDURE:

1. Import NumPy, Matplotlib, and the required Scikit-learn modules.
2. Generate study-hour data and nonlinear examination marks with random noise.
3. Split the dataset into training and testing sets.
4. Train a linear regression model using the training data.
5. Calculate training and testing mean squared errors.
6. Plot actual marks and predicted values to identify possible underfitting.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
np.random.seed(42)
hours = np.linspace(0, 10, 80)
marks = 10 + 6 * hours + 5 * np.sin(hours) \
        + np.random.normal(0, 2, 80)
X = hours.reshape(-1, 1)
y = marks
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
model = LinearRegression()
model.fit(X_train, y_train)
train_pred = model.predict(X_train)
test_pred = model.predict(X_test)
train_mse = mean_squared_error(y_train, train_pred)
test_mse = mean_squared_error(y_test, test_pred)
print("Training MSE:", round(train_mse, 2))
print("Testing MSE:", round(test_mse, 2))
```

```
x_line = np.linspace(0, 10, 200).reshape(-1, 1)

y_line = model.predict(x_line)

plt.scatter(X, y, label="Actual Marks")

plt.plot(x_line, y_line, label="Linear Prediction")

plt.xlabel("Study Hours")

plt.ylabel("Examination Marks")

plt.title("Linear Regression: Student Performance")

plt.legend()

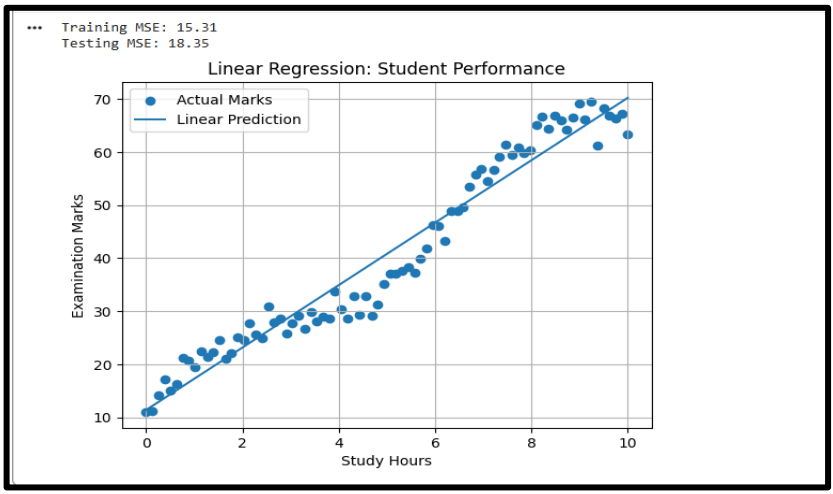
plt.grid(True)

plt.show()
```

INPUT:

- Study hours: 80 generated values between 0 and 10.
- Marks: A nonlinear function of study hours with random noise.
- Model: Linear regression.
- Test size: 20%.

OUTPUT:



RESULT:

The linear regression model was trained and evaluated successfully. Because the generated data contain a nonlinear pattern, a straight-line model may underfit the data, producing errors that could be reduced by using a suitable nonlinear model.

3. A deep learning research team uses a machine learning model to classify images based on extracted numerical features. A highly complex polynomial model performs well on training data but poorly on unseen testing data.

Write a Python program to generate a sample nonlinear dataset, train polynomial regression models with degrees 2 and 15, calculate their training and testing errors, and to compare their fitted curves.

AIM:

To generate a nonlinear dataset, train polynomial regression models of degrees 2 and 15, calculate their training and testing errors, and compare their fitted curves to study overfitting.

PROCEDURE:

1. Generate nonlinear sample data using a sine function with random noise.
2. Divide the data into training and testing sets.
3. Train polynomial regression models of degrees 2 and 15.
4. Calculate training and testing MSE for both models.
5. Generate predictions over a smooth range of input values.
6. Plot the fitted curves and compare model performance.

PROGRAM CODE:

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.pipeline import make_pipeline

from sklearn.preprocessing import PolynomialFeatures, StandardScaler
```

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
np.random.seed(42)
X = np.sort(np.random.uniform(0, 10, 35)).reshape(-1, 1)
y = np.sin(X[:, 0]) + np.random.normal(0, 0.25, 35)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)
x_curve = np.linspace(0, 10, 300).reshape(-1, 1)
plt.figure(figsize=(10, 6))
plt.scatter(X_train, y_train, label="Training Data")
plt.scatter(X_test, y_test, label="Testing Data")
for degree in [2, 15]:
    model = make_pipeline(
        PolynomialFeatures(degree=degree, include_bias=False),
        StandardScaler(),
        LinearRegression()
    )
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    print(f"\nPolynomial Degree {degree}")
```

```
    print("Training MSE:",
          round(mean_squared_error(y_train, train_pred), 4))
    print("Testing MSE:",
          round(mean_squared_error(y_test, test_pred), 4))
    plt.plot(x_curve, model.predict(x_curve),
             label=f"Degree {degree}")
plt.xlabel("Input Feature")
plt.ylabel("Target Value")
plt.title("Underfitting vs Overfitting")
plt.legend()
plt.grid(True)
plt.ylim(-3, 3)
plt.show()
```

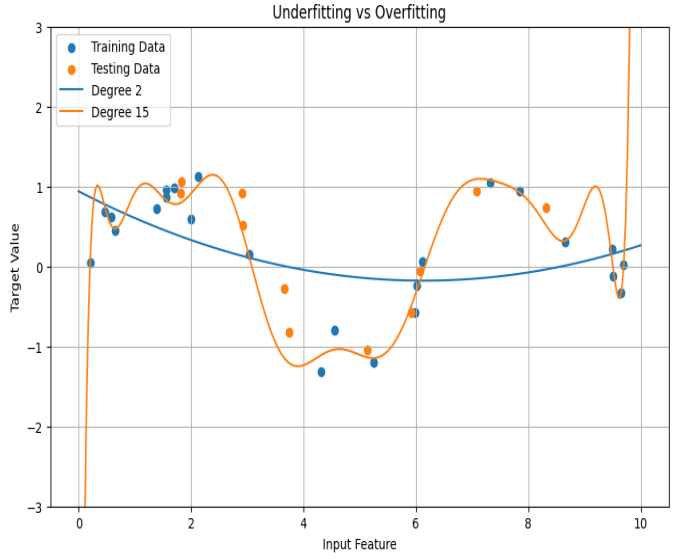
INPUT:

- Dataset: 35 randomly generated nonlinear observations.
- Target: Sine function with added noise.
- Polynomial degrees: 2 and 15.
- Test size: 30%.

OUTPUT:

```
Polynomial Degree 2
Training MSE: 0.3596
Testing MSE: 0.4576

Polynomial Degree 15
Training MSE: 0.0187
Testing MSE: 0.1209
```



RESULT:

The degree-2 model may underfit the oscillating data because its curve is relatively simple. The degree-15 model is more flexible and may fit the training observations closely while producing unstable predictions on unseen data. A substantially lower training error than testing error is evidence of possible overfitting.

4. A weather forecasting team develops prediction models with different polynomial degrees to estimate temperature based on time-related features. The team wants to identify a suitable model that balances simplicity and predictive accuracy.

Write a Python program to generate sample temperature data, train polynomial regression models with degrees 1, 3, and 12, calculate the training and testing mean squared errors, and to compare model performance.

AIM:

To generate sample temperature data, train polynomial regression models of degrees 1, 3, and 12, calculate training and testing mean squared errors, and identify a model that balances simplicity and predictive accuracy.

PROCEDURE:

1. Generate temperature observations over a 24-hour period with random noise.
2. Split the dataset into training and testing sets.
3. Train polynomial regression models of degrees 1, 3, and 12.
4. Calculate training and testing MSE for each model.
5. Plot the prediction curves to compare the models.
6. Select a suitable model based on its testing performance and complexity.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler, PolynomialFeatures
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_squared_error
np.random.seed(42)
time = np.linspace(0, 24, 100)
temperature = (
    20 + 8 * np.sin(2 * np.pi * time / 24)
    + np.random.normal(0, 1.5, 100)
)
X = time.reshape(-1, 1)
y = temperature
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
x_curve = np.linspace(0, 24, 300).reshape(-1, 1)
plt.figure(figsize=(10, 6))
plt.scatter(X, y, label="Actual Temperature")
```

```
for degree in [1, 3, 12]:
    model = make_pipeline(
        StandardScaler(),
        PolynomialFeatures(degree=degree, include_bias=False),
        LinearRegression()
    )
    model.fit(X_train, y_train)
    train_pred = model.predict(X_train)
    test_pred = model.predict(X_test)
    print(f"\nPolynomial Degree {degree}")
    print("Training MSE:",
          round(mean_squared_error(y_train, train_pred), 2))
    print("Testing MSE:",
          round(mean_squared_error(y_test, test_pred), 2))
    plt.plot(x_curve, model.predict(x_curve),
             label=f"Degree {degree}")
plt.xlabel("Time (hours)")
plt.ylabel("Temperature (°C)")
plt.title("Temperature Prediction Using Polynomial Regression")
plt.legend()
plt.grid(True)
plt.show()
```

INPUT:

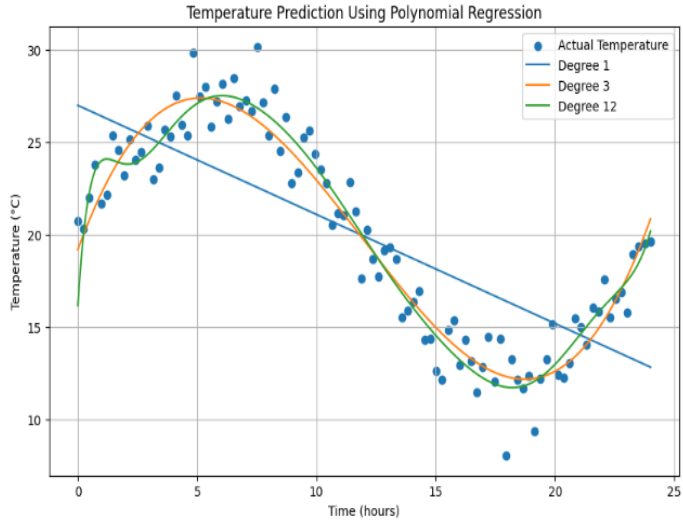
- Time values: 100 observations between 0 and 24 hours.
- Temperature: A sinusoidal function with random noise.
- Polynomial degrees: 1, 3, and 12.
- Test size: 20%.

OUTPUT:

```
Polynomial Degree 1
Training MSE: 12.71
Testing MSE: 12.96

Polynomial Degree 3
Training MSE: 2.18
Testing MSE: 1.63

Polynomial Degree 12
Training MSE: 1.66
Testing MSE: 2.74
```



RESULT:

The degree-1 model underfits the nonlinear temperature pattern. The degree-3 model achieves the lowest testing MSE of approximately 1.63, while the degree-12 model has a lower training error but a higher testing error. For this dataset, degree 3 provides the best balance between model complexity and predictive accuracy among the three models.

5. An online learning platform predicts student assessment scores using a machine learning model. The development team wants to understand whether the model suffers from hig bias or high variance by examining its performance as the amount of training data increases.

Write a Python program to generate a sample regression dataset, calculate learning curves for a linear regression model and a high-degree polynomial regression model, and to plot training and validation errors against the number of training samples.

AIM:

To generate a sample regression dataset, calculate learning curves for a linear regression model and a high-degree polynomial regression model, and plot training and validation errors against the number of training samples.

PROCEDURE:

1. Generate a nonlinear regression dataset with random noise.
2. Define a linear regression model and a degree-10 polynomial regression model.
3. Use Scikit-learn's learning_curve() function to calculate training and validation errors at different training sizes.
4. Convert negative mean squared error scores into positive MSE values.
5. Plot training and validation errors against the number of training samples for both models.
6. Compare the error patterns to identify possible high bias or high variance.

PROGRAM CODE:

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_regression
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import PolynomialFeatures, StandardScaler
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import learning_curve
np.random.seed(42)
X = np.linspace(0, 10, 150).reshape(-1, 1)
y = 2 * X[:, 0] + np.sin(2 * X[:, 0]) \
   + np.random.normal(0, 1, 150)
models = {
   "Linear Regression": LinearRegression(),
   "Polynomial Degree 10": make_pipeline(
      StandardScaler(),
      PolynomialFeatures(degree=10, include_bias=False),
      LinearRegression()
   )
}
train_sizes = np.linspace(0.1, 1.0, 8)
fig, axes = plt.subplots(1, 2, figsize=(14, 5))
```

```
for ax, (name, model) in zip(axes, models.items()):
    sizes, train_scores, val_scores = learning_curve(
        model,
        X,
        y,
        train_sizes=train_sizes,
        cv=5,
        scoring="neg_mean_squared_error",
        shuffle=True,
        random_state=42
    )
    train_mse = -train_scores.mean(axis=1)
    val_mse = -val_scores.mean(axis=1)
    print(f"\n{name}")
    print("Training MSE:", np.round(train_mse, 3))
    print("Validation MSE:", np.round(val_mse, 3))
    ax.plot(sizes, train_mse, marker="o",
            label="Training MSE")
    ax.plot(sizes, val_mse, marker="s",
            label="Validation MSE")
    ax.set_xlabel("Number of Training Samples")
    ax.set_ylabel("Mean Squared Error")
    ax.set_title(name)
```

```
    ax.legend()
    ax.grid(True)
plt.suptitle("Learning Curves: Bias and Variance Analysis")
plt.tight_layout()
plt.show()
```

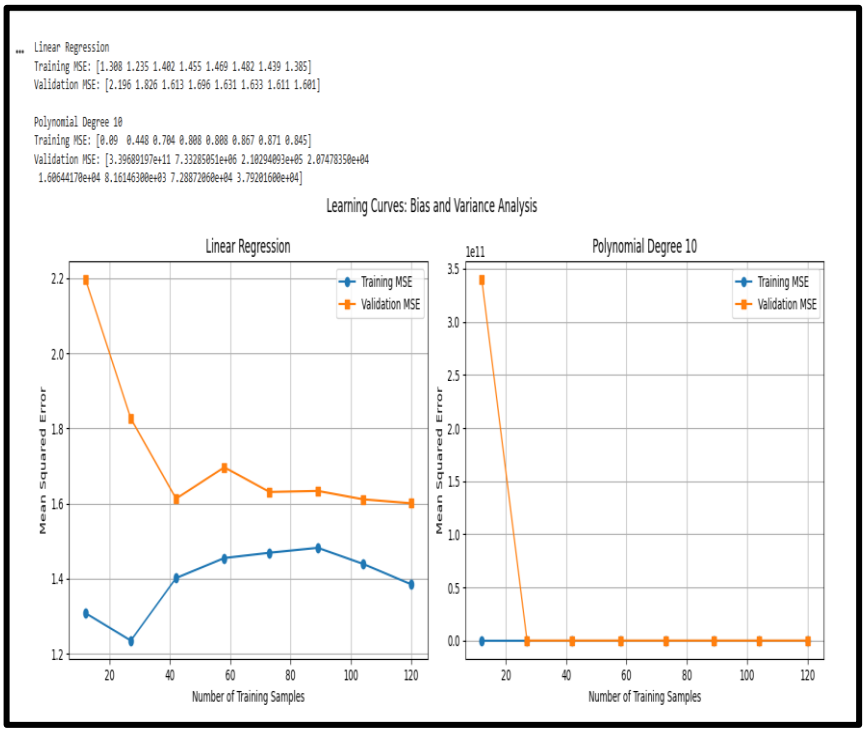
INPUT:

- Dataset: 150 generated observations.
- Target function: A nonlinear relationship with random noise.
- Models: Linear regression and degree-10 polynomial regression.
- Cross-validation: 5-fold.
- Training sizes: 8 different sample sizes.

OUTPUT:

```
Linear Regression
Training MSE: [1.308 1.235 1.402 1.455 1.469 1.482 1.439 1.385]
Validation MSE: [2.196 1.826 1.613 1.696 1.631 1.633 1.611 1.601]

Polynomial Degree 10
Training MSE: [0.09  0.448 0.704 0.808 0.808 0.867 0.871 0.845]
Validation MSE: [3.39689197e+11 7.33285051e+06 2.10294093e+05 2.07478350e+04
 1.60644170e+04 8.16146300e+03 7.28872060e+04 3.79201600e+04]
```



RESULT:

The learning curves were generated successfully for both models. They show how prediction errors change as more training data become available. The linear model may have difficulty capturing the nonlinear relationship, while the high-degree polynomial model may be sensitive to the training samples. Increasing training data can help reduce variance, although it does not necessarily eliminate underfitting.